

Resumen De Entrega - Fase 1 Backend

Resumen Ejecutivo

Quedo listo el backend inicial completo de la fase 1 en un monorepo NestJS con TypeScript, PostgreSQL en Supabase, Swagger, JWT, class-validator, variables de entorno y arquitectura modular por servicios.

Se implementaron estos servicios:

- **api-gateway**
- **auth-service**
- **access-control-service**
- **institution-service**
- **identity-service**

No se implementaron aun:

- **academic-structure-service**
- **enrollment-service**
- **audit-service**
- frontend
- app movil

Decision De ORM

Se eligio **TypeORM**.

Justificacion breve: la base de datos ya existia, el dominio estaba congelado y en esta fase era mas seguro mapear explicitamente tablas, columnas, UUIDs y relaciones reales del esquema existente sin rediseñarlo ni generar un esquema alternativo.

Que Quedo Listo

- monorepo NestJS estructurado por apps y libs
- integracion real con la base existente en Supabase
- Swagger / OpenAPI en los servicios expuestos
- **JwtAuthGuard, RolesGuard e InstitutionContextGuard**
- access token y refresh token
- login por correo y contraseña
- cambio obligatorio de contraseña en primer ingreso
- logout
- recuperacion y restablecimiento de contraseña
- manejo de sesiones

- control de intentos fallidos
- preparacion para login con Google / OIDC
- CRUD institucional de la fase 1
- CRUD base de personas y catalogos
- control de acceso por institucion
- asignacion de perfiles, roles y administradores institucionales

Reglas De Negocio Respetadas

- plataforma multiinstitucion
- una persona puede existir una sola vez en el sistema
- una persona puede tener varios usuarios
- cada usuario pertenece a una sola institucion
- excepcion: superadministrador global sin institucion
- ningun usuario institucional puede operar fuera de su institucion
- solo el superadministrador puede crear instituciones
- administrador institucional no puede crear instituciones
- solo perfiles funcionales permitidos pueden recibir el rol de administrador institucional
- la contraseña inicial se maneja como hash y el primer ingreso obliga cambio

Endpoints Implementados Y Validados

Auth

- POST /auth/login
- POST /auth/refresh
- POST /auth/logout
- POST /auth/solicitar-recuperacion
- POST /auth/restablecer-contrasena
- POST /auth/cambiar-contrasena-inicial
- GET /auth/sesiones

Access Control

- GET /usuarios/:id/contexto-acceso
- GET /usuarios/:id/permisos-efectivos
- POST /usuarios/:id/perfiles
- POST /usuarios/:id/roles
- POST /instituciones/:id/administradores-app

Institution

- **POST /instituciones**
- **GET /instituciones**
- **GET /instituciones/:id**
- **PATCH /instituciones/:id**
- **POST /instituciones/:id/sedes**
- **GET /instituciones/:id/sedes**
- **POST /instituciones/:id/anos-lectivos**
- **GET /instituciones/:id/anos-lectivos**
- **POST /anos-lectivos/:id/periodos**
- **GET /anos-lectivos/:id/periodos**
- **POST /periodos/:id/cerrar**
- **PUT /instituciones/:id/configuracion**
- **GET /instituciones/:id/configuracion**
- **POST /instituciones/:id/escalas-valoracion**
- **GET /instituciones/:id/escalas-valoracion**

Identity

- **POST /personas**
- **GET /personas/:id**
- **PATCH /personas/:id**
- **GET /personas/buscar-por-documento**
- **GET /tipos-documento**
- **GET /generos**

Resumen Tecnico Por Carpetas Y Archivos Clave

Raiz Del Proyecto

- **package.json**: scripts de build y arranque del monorepo
- **nest-cli.json**: configuracion del monorepo Nest
- **tsconfig.json** y **tsconfig.build.json**: compilacion TypeScript
- **.env.example**: plantilla de variables de entorno
- **README.md**: arranque, arquitectura y validacion real

apps/api-gateway

- **apps/api-gateway/src/main.ts**: bootstrap del gateway en puerto configurable
- **apps/api-gateway/src/api-gateway.module.ts**: compone los modulos funcionales, carga **ConfigModule**, **DatabaseModule**, **AuthSupportModule** y registra guards globales

Responsabilidad tecnica:

- punto unico de entrada REST
- aplicacion global de seguridad
- composicion de servicios internos

apps/auth-service

- **apps/auth-service/src/auth-service.module.ts**: modulo del servicio de autenticacion
- **apps/auth-service/src/auth.controller.ts**: expone login, refresh, logout, recuperacion, cambio inicial y sesiones
- **apps/auth-service/src/auth.service.ts**: implementa la logica principal de autenticacion, sesiones y recuperacion
- **apps/auth-service/src/dto.ts**: DTOs y validaciones del modulo

Responsabilidad tecnica:

- resolver login por correo e institucion
- validar hash de contrasena
- emitir access y refresh token
- exigir cambio de contrasena cuando aplica
- registrar sesiones
- revocar sesiones en logout
- generar y consumir tokens de recuperacion

apps/access-control-service

- **apps/access-control-service/src/access-control-service.module.ts**: modulo del servicio
- **apps/access-control-service/src/access-control.controller.ts**: endpoints de contexto, permisos, perfiles, roles y administrador institucional
- **apps/access-control-service/src/access-control.service.ts**: logica de perfiles, roles, permisos efectivos y reglas de administracion
- **apps/access-control-service/src/dto.ts**: DTOs de asignacion

Responsabilidad tecnica:

- construir contexto de acceso
- calcular permisos efectivos
- asignar perfiles funcionales
- asignar roles tecnicos
- aplicar reglas del superadministrador unico
- validar quien puede ser administrador institucional

apps/institution-service

- **apps/institution-service/src/institution-service.module.ts**: modulo institucional
- **apps/institution-service/src/institution.controller.ts**: expone instituciones, sedes, anios lectivos, periodos, configuracion y escalas
- **apps/institution-service/src/institution.service.ts**: implementa CRUD y validacion de contexto institucional
- **apps/institution-service/src/dto.ts**: DTOs y validaciones de escritura

Responsabilidad tecnica:

- CRUD de instituciones
- CRUD de sedes
- gestion de anios lectivos y periodos
- cierre de periodos
- configuracion institucional
- escalas de valoracion con niveles

apps/identity-service

- **apps/identity-service/src/identity-service.module.ts**: modulo de identidad
- **apps/identity-service/src/identity.controller.ts**: endpoints de personas y catalogos
- **apps/identity-service/src/identity.service.ts**: logica CRUD de personas y consulta por documento
- **apps/identity-service/src/dto.ts**: DTOs y validaciones del modulo

Responsabilidad tecnica:

- crear personas sin duplicar documento
- consultar por id
- actualizar datos basicos
- buscar por tipo de documento y numero
- exponer catalogos de tipos de documento y generos

libs/common

- **libs/common/src/bootstrap.ts**: configuracion comun de Nest, Swagger, pipes, filtros e interceptores
- **libs/common/src/constants/auth.constants.ts**: constantes de roles, perfiles y metadata keys
- **libs/common/src/decorators/current-user.decorator.ts**: acceso al payload autenticado
- **libs/common/src/decorators/public.decorator.ts**: marca rutas publicas
- **libs/common/src/decorators/roles.decorator.ts**: metadata de roles requeridos
- **libs/common/src/decorators/institution-scoped.decorator.ts**: metadata de contexto institucional
- **libs/common/src/guards/jwt-auth.guard.ts**: guard de autenticacion JWT
- **libs/common/src/guards/roles.guard.ts**: guard de roles
- **libs/common/src/guards/institution-context.guard.ts**: guard de alcance institucional

- **libs/common/src/exceptions/api-exception.filter.ts**: respuesta uniforme de errores
- **libs/common/src/interceptors/response-envelope.interceptor.ts**: envelope estandar de respuestas
- **libs/common/src/types/jwt-payload.type.ts**: contrato del payload JWT

libs/database

- **libs/database/src/database.module.ts**: configuracion TypeORM contra Supabase usando **DATABASE_URL**
- **libs/database/src/base.entity.ts**: base comun para entidades con UUID y auditoria
- **libs/database/src/entities.ts**: entidades mapeadas a tablas reales del esquema existente
- **libs/database/src/index.ts**: exportaciones del modulo de base de datos

Responsabilidad tecnica:

- conexion a PostgreSQL/Supabase
- carga de entidades reales
- **synchronize: false** para no modificar la base
- alineacion con columnas reales como **creado_en**, **actualizado_en**, **hash_contrasena**, **numero_secuencia**, **tipo_escalavaloracion**, etc.

libs/auth

- **libs/auth/src/auth-support.module.ts**: modulo compartido de soporte auth
- **libs/auth/src/password.service.ts**: hashing y comparacion de contraseñas
- **libs/auth/src/token.service.ts**: emision y verificacion de JWT access/refresh
- **libs/auth/src/index.ts**: exportaciones del modulo

libs/contracts

- **libs/contracts/src/index.ts**: espacio reservado para contratos compartidos entre servicios

libs/testing

- **libs/testing/src/index.ts**: espacio preparado para utilidades de testing futuras

Seguridad Y Hardening Aplicado

- JWT obligatorio en rutas protegidas
- roles verificados por guard global
- alcance institucional verificado por guard y por servicio cuando la relacion no venia directa en la URL
- validacion de UUID en rutas criticas
- validacion de DTOs con **whitelist**, **forbidNonWhitelisted** y transformacion automatica

- backend alineado con los codigos reales del dominio: **superadministrador**, **administrador_app_institucion**, **docente_directivo**

Entregables Finales

- estructura completa del monorepo
- configuracion inicial del proyecto
- codigo de los 5 servicios iniciales
- modulos, controladores, servicios, DTOs y guards
- integracion con base de datos existente
- documentacion Swagger
- archivos de entorno de ejemplo
- README de arranque y validacion real

Estado Final

La fase 1 quedo funcional y validada contra la base real de Supabase, sin modificar la estructura de la base de datos y dejando el proyecto preparado para continuar con fases posteriores.